CZĘŚĆ 1 · MODUŁ W CZTERECH CZĘŚCIACH

GIT W PRACY Z AGENTEM AI

Wprowadzenie do wersjonowania i bezpiecznego rozwijania własnych projektów

Git zapisuje historię projektu.

GitHub przechowuje tę historię poza komputerem.

Agent zmienia pliki, a człowiek zatwierdza dobre wersje.

ROBERT MATYSZEWSKI

HELLO!
CO TAM U WAS?

01

Nad czym pracowałeś?

02

Co się udało?

03

Co było wyzwaniem?

CZĘŚĆ I

CO TO JEST GIT I GITHUB?

01 Problem: agent zmienia pliki szybko

02 Czym jest wersjonowanie

03 Skąd się wziął Git

04 Git a GitHub

05 Commit i push

06 Co trafia do repozytorium

CEL SZKOLENIA

TAK WYGLĄDA CHAOS

```
moj-projekt/
├── oferta.md
├── oferta-final.md
├── oferta-finalv2.md
├── oferta-finalv2-poprawki.md
└── oferta-OSTATECZNA.md
```

PO TYM MODULE UCZESTNIK POTRAFI

01 wyjaśnić, czym jest wersjonowanie i po co powstało

02 wyjaśnić różnicę między Gitem i GitHubem

03 zamienić zwykły folder projektu w repozytorium

04 utworzyć prywatne repozytorium na GitHubie

05 zlecić agentowi status, commit i push

06 poprosić agenta o odrzucenie nieudanej zmiany (restore)

07 używać tych komend albo skilla — bez uczenia się ich na pamięć

AGENT ZMIENIA PLIKI SZYBCIEJ, NIŻ CZŁOWIEK JE KONTROLUJE

Bez historii projektu trudno odpowiedzieć na dwa pytania:

Co dokładnie się zmieniło?

Jak wrócić do dobrej wersji?

CZYM JEST WERSJONOWANIE

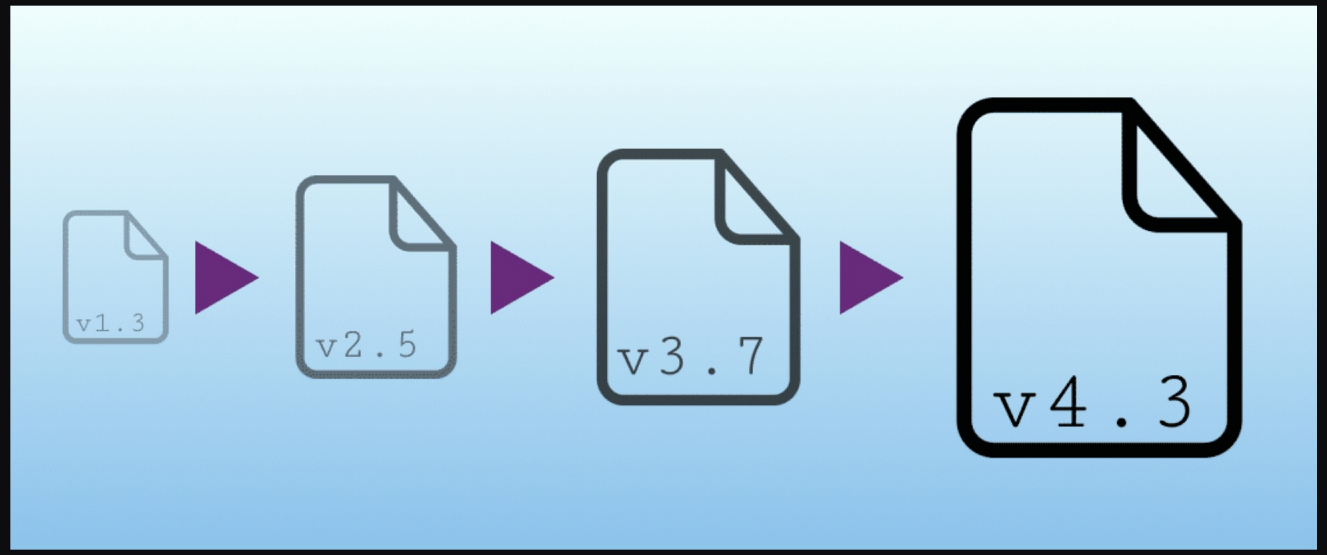
Wersjonowanie to zapisywanie zmian w plikach w taki sposób, aby można było odtworzyć wcześniejszy stan.

wiadomo, co się zmieniło;

wiadomo, kiedy powstała zmiana;

można nazwać ważne wersje;

można wrócić do wcześniejszego stanu.



PO CO POWSTAŁA KONTROLA WERSJI

Bez systemu wersjonowania często powstają pliki:

```
agent-final
agent-final-2
agent-final-naprawde
agent-final-naprawde-ostateczny
```

System kontroli wersji zastępuje ręczne kopie uporządkowaną historią.

SOFTWARE JAK BIERKI

Każdy projekt to wiele połączonych elementów.

Ruszasz jeden plik — może przesunąć się coś innego.

Im większy projekt, tym trudniej przewidzieć skutki zmiany.

Agent potrafi ruszyć wiele bierek naraz.

Dlatego potrzebujemy narzędzia, które pokaże, co się ruszyło, i pozwoli cofnąć ruch.



DLACZEGO POWSTAŁ GIT

Git powstał w 2005 roku na potrzeby rozwoju systemu Linux.

Projekt potrzebował narzędzia, które było:

szybkie;

rozproszone;

wydajne przy bardzo dużym projekcie;

przygotowane na tysiące równoległych gałęzi.



Linus Torvalds

CZYM JEST GIT

Git to rozproszony system kontroli wersji.

Git:

- rozpoznaje zmiany w plikach;
- zapisuje wybrane wersje jako commity;
- przechowuje historię projektu;
- pozwala odzyskiwać wcześniejszy stan.

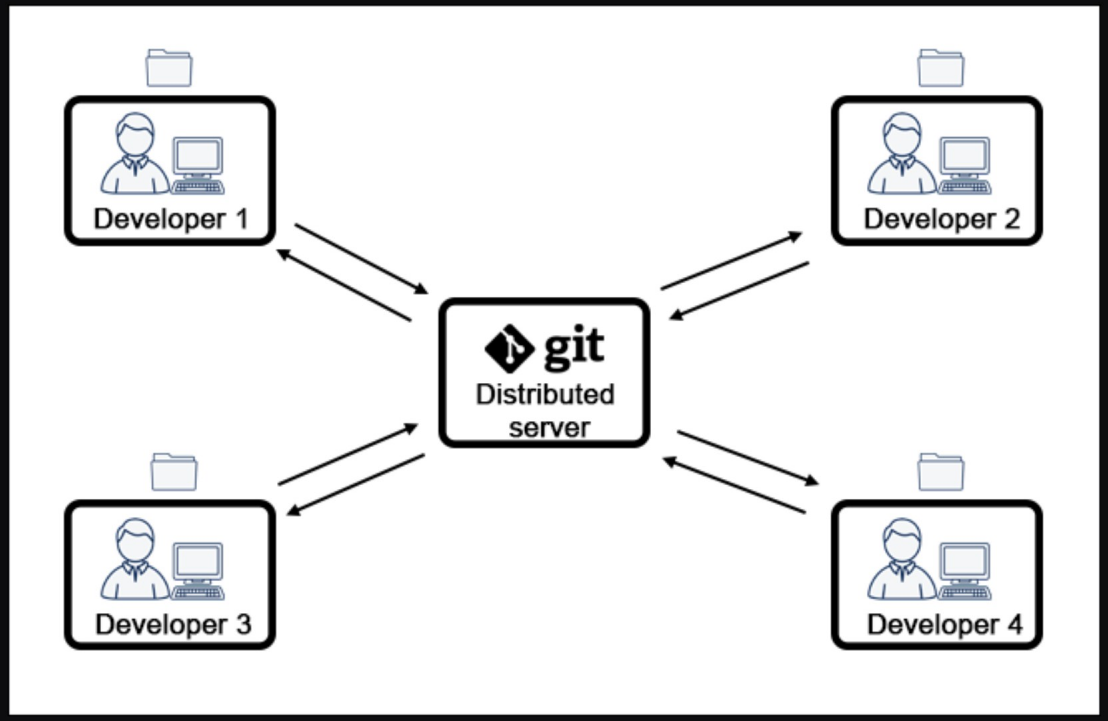


CO ZNACZY „ROZPROSZONY”

W Gicie pełna historia może istnieć w wielu miejscach:

```
komputer główny ↔ GitHub ↔ drugi komputer
```

Każdy pełny klon repozytorium zawiera pliki i historię.

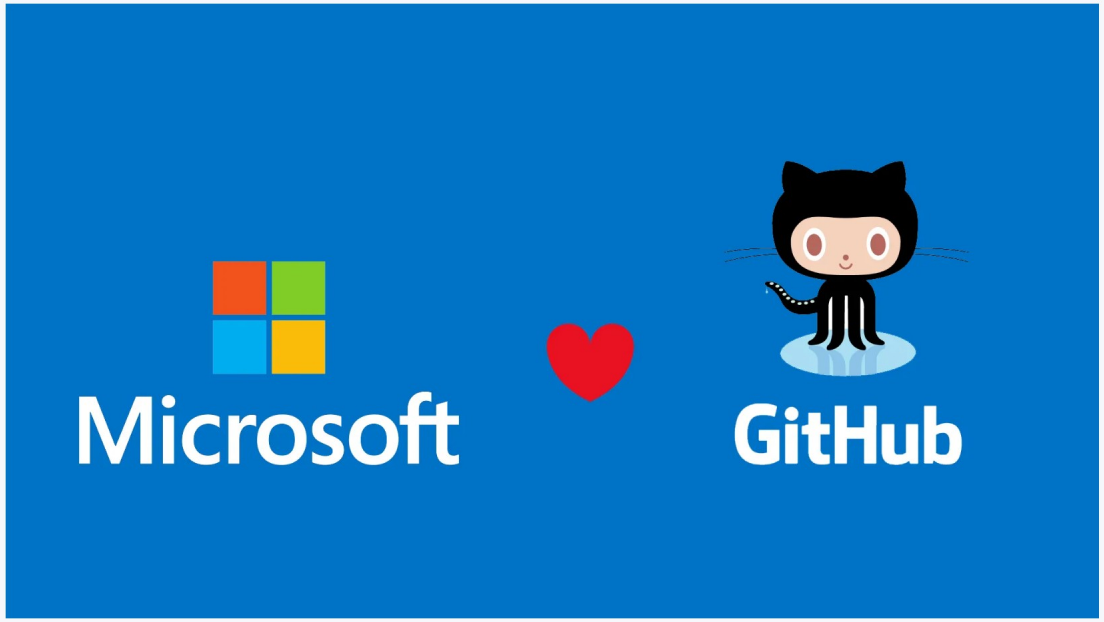


CZYM JEST GITHUB

GitHub to internetowa platforma zbudowana wokół repozytoriów Git.

GitHub umożliwia:

- przechowywanie wypchniętych commitów;
- zarządzanie dostępem do projektu;
- proponowanie i przeglądanie zmian;
- prowadzenie zadań, dyskusji i automatyzacji.



CO GITHUB MA WSPÓLNEGO Z PROGRAMOWANIEM

PROGRAMIŚCI WYKORZYSTUJĄ GITHUB DO:

01
wspólnej pracy nad kodem

02
zgłaszania błędów i planowania funkcji

03
przeglądania zmian przed połączeniem

04
uruchamiania testów i procesów publikacji

Git przechowuje historię plików. GitHub organizuje pracę wokół tej historii.

GIT A GITHUB — CZYM SIĘ RÓŻNIĄ

	Git	GitHub
Czym jest	system kontroli wersji	platforma internetowa
Gdzie działa	na Twoim komputerze	w internecie
Czy wymaga internetu	nie, zapisuje historię lokalnie	tak
Co robi	zapisuje zmiany jako commity i przechowuje historię	przechowuje wypchnięte commity poza komputerem
Co dodaje	—	dostęp do projektu, przegląd zmian, zadania, automatyzacje

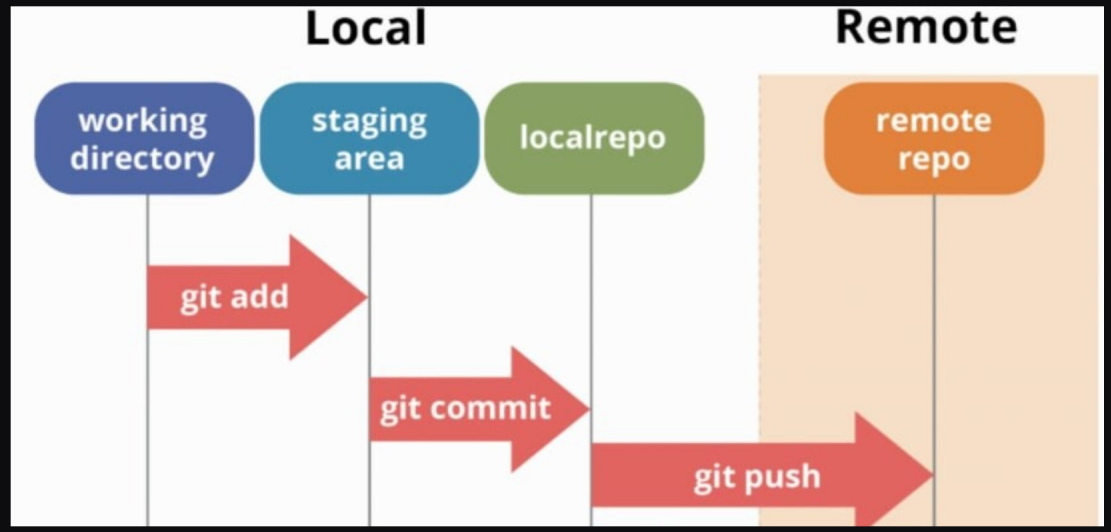
Git działa bez GitHuba. GitHub nie działa bez Gita.

NAJWAŻNIEJSZE POJĘCIA

Pojęcie	Co znaczy
Repozytorium (REPO)	folder projektu z historią Gita
Status	lista tego, co się zmieniło od ostatniego commita
Commit	nazwany punkt kontrolny — zapisana wersja projektu
Push	wysłanie commitów (wersji projektu) na GitHub
Restore	odrzucenie niezapisanej zmiany
Remote	adres repozytorium na GitHubie
.gitignore	lista plików, których Git ma nie zapisywać

COMMIT I PUSH TO DWIE RÓŻNE CZYNNOŚCI

`commit`	zapisuje checkpoint lokalnie
`push`	wysyła istniejące commity na GitHub



Zapisanie pliku nie tworzy commita. Commit nie wykonuje pushu.

CO WRZUCAM DO REPOZYTORIUM

01 **Kod, treści i potrzebne obrazy**
`.html .css .js .py .md, logo`

02 **Wymagania techniczne skryptów**
`package.json, requirements.txt`

03 **Instrukcje dla agenta**
`AGENTS.md, CLAUDE.md`

04 **Opis projektu**
`README.md`

05 **Wzory ustawień bez sekretów**
`.gitignore, .env.example`

06 **Testy i skrypty**
`tests/, scripts/`

Commituję wszystko, czego trzeba, żeby projekt odtworzyć i dalej rozwijać.

CO WYKLUCZAM W PLIKU .GITIGNORE

01 **Hasła i klucze dostępu**
plik .env — nie może trafić do historii

02 **Pobrane biblioteki**
agent pobierze je ponownie jednym poleceniem

03 **Pliki tymczasowe**
śmieci systemu i programów, np. .DS_Store

04 **Gotowe eksporty**
eksporty PDF i zbudowana strona — da się je wygenerować ponownie

05 **Lokalne bazy danych**
*.sqlite, *.db

06 **Ciężkie pliki**
nagrania, backupy, archiwa

.gitignore nie działa na plik, który trafił już do commita.

PODSUMOWANIE

01 STATUS
sprawdzam, co się zmieniło

02 COMMIT
zapisuję dobrą wersję

03 PUSH
wysyłam commity na GitHub

Zła zmiana przed commitem? Restore.



CZĘŚĆ II

PODSTAWOWA PĘTLA PRACY

01	STATUS: co jest w folderze	02	Jedna mała zmiana	03	COMMIT: dobra wersja
04	PUSH: checkpoint na GitHubie	05	RESTORE: nieudany eksperyment	06	Minimum bezpieczeństwa

JEDNA PĘTLA DO ZAPAMIĘTANIA

01

Zmiana projektu

zleć agentowi jedną małą zmianę

02

STATUS

sprawdź stan projektu

03

COMMIT

zapisz dobrą wersję

04

PUSH

wyślij checkpoint na GitHub

05

RESTORE

użyj tylko wtedy, gdy zmiana jest zła i nie było commita

RESTORE to droga awaryjna, nie stały krok pętli.

STATUS: CO ZNAJDUJE SIĘ W FOLDERZE

TAK TO WIDAĆ U AGENTA I W EDYTORZE

```
main · ahead 2
M  index.html
A  styles.css
D  stary-baner.png
R  logo.svg → logo-nowe.svg
?? obraz.png
```

CO ZNACZĄ OZNACZENIA

M	zmieniony plik, który Git już zna
A	nowy plik dodany do commita
D	usunięty plik
R	zmieniona nazwa pliku
??	nowy plik, którego Git jeszcze nie śledzi
ahead 2	2 lokalne commity, których GitHub jeszcze nie ma — czas na push

Komenda: git status · Prompt do agenta: „Zrób git status.”

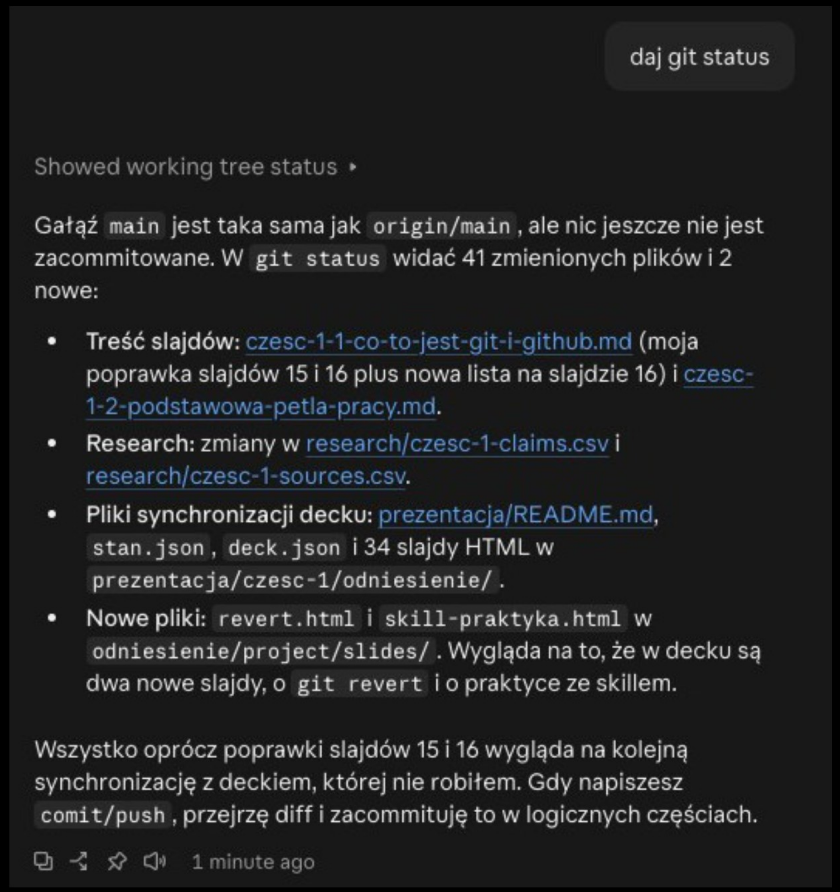
STATUS W APLIKACJI CLAUDE

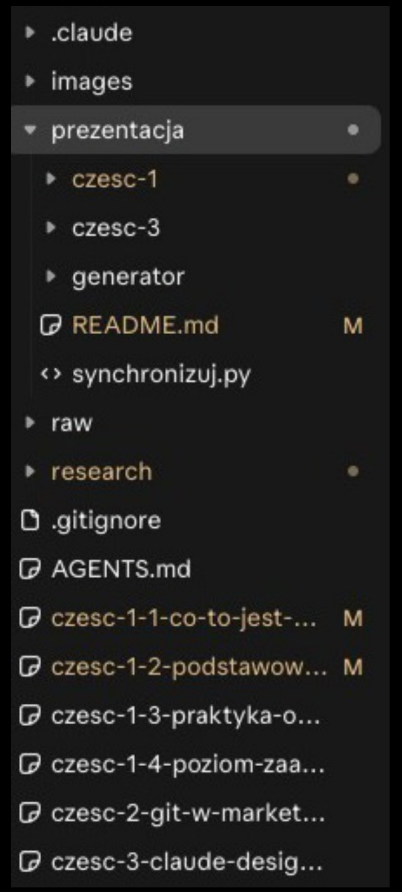
W ROZMOWIE

Piszę „daj git status”. Agent tłumaczy wynik prostym językiem.

W DRZEWIE PLIKÓW

Litera M przy zmienionych plikach, kropka przy folderach ze zmianami.





ZLECAMY JEDNĄ MAŁĄ ZMIANĘ

PRZYKŁADOWY PROMPT

„Dodaj nowe obrazek do slajdu 21 do prezentacji.”

Zakres plików

tylko pliki prezentacji

Bez commita i pushu

najpierw podgląd i status

COMMIT: ZAPISUJEMY DOBRĄ WERSJĘ

Commit zapisuje wybrane pliki jako nazwany punkt kontrolny.

KOMENDY DO ROZPOZNANIA

`git add` `git commit`

PROMPT DO AGENTA

„Pokaż status"

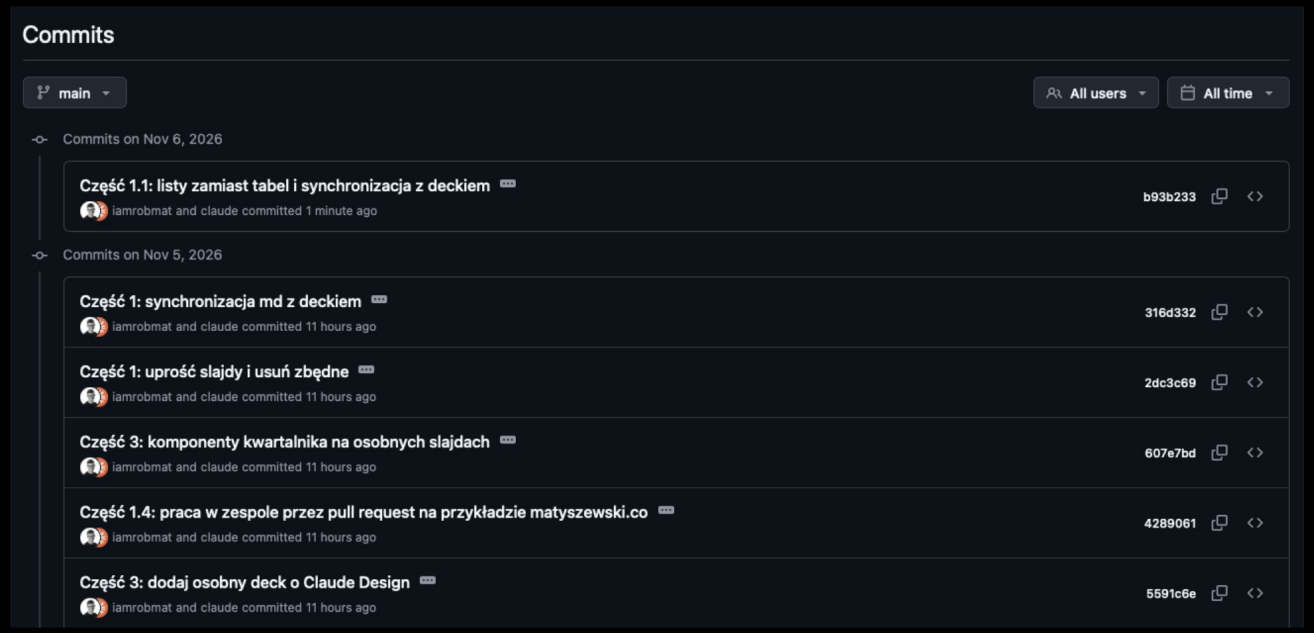
PUSH: WYSYŁAMY ZMIANY NA GITHUB

Push wysyła zapisane commity na GitHub.

Niezacommitowane zmiany zostają na komputerze.

KOMENDY DO ROZPOZNANIA

`git log` `git push`



PROMPT DO AGENTA

„Sprawdź, czy ostatni commit to »Dodaj nowe slajdy do prezentacji«, a status nie pokazuje nieoczekiwanych zmian. Jeżeli wszystko się zgadza, wykonaj git push i potwierdź gałąź docelową.”

RESTORE: ODRZUCAMY NIEUDANY EKSPERYMENT

Warunek: zmiana jest zła i nie było jeszcze commita.

Pliki wracają do stanu z ostatniego commita.

KOMENDY DO ROZPOZNANIA

```
git restore
```

PROMPT DO AGENTA

„Odrzuć wyłącznie niezapisane zmiany w index.html i styles.css, przywracając te pliki z ostatniego commita. Nie zmieniaj innych plików. Po operacji pokaż git status.”

REVERT: COFAMY ZAPISANY COMMIT

Warunek: zła zmiana jest już w commicie.

Revert dodaje nowy commit, który odwraca zmiany. Stary commit zostaje w historii.

KOMENDY DO ROZPOZNANIA

`git log` `git revert`

PROMPT DO AGENTA

„Pokaż ostatnie commity. Cofnij commit »Dodaj nowe slajdy do prezentacji« przez git revert. Nie usuwaj historii. Po operacji pokaż git log i git status.”

TRZY SYTUACJE, TRZY DECYZJE

Sytuacja A

Zmiana wygląda dobrze. Status pokazuje zmienione oczekiwane pliki

Decyzja: commit konkretnych plików, potem push.

Sytuacja B

Zmiana wygląda źle. Nie było commita.

Decyzja: restore wskazanych plików.

Sytuacja C

Status pokazuje zmiany i pliki, których nie rozpoznajemy.

Decyzja: stop. Agent najpierw wyjaśnia stan.

MÓJ SKRÓT: SKILL COMMIT

Kiedy zmiana jest gotowa, wpisuję agentowi: **comit**. Skill to plik z instrukcją, którą agent wykonuje krok po kroku:

01	sprawdza, co się zmieniło
02	opisuje każdy plik
03	pisze opis commita po polsku
04	robi commit i push
05	podaje hash commita

SKILL.MD — TREŚĆ SKILLA

```
---
name: comit description: analizuje zmiany, pisze opis
commita po polsku, robi commit i push
---
1. Sprawdź status zmian git status --short, git diff --
   stat Brak zmian? Powiedz to i zakończ.
2. Przeanalizuj każdą zmianę nowe, zmienione i usunięte
   pliki pomiń .DS_Store
3. Napisz tytuł i opis po polsku tytuł do ok. 72 znaków
   nowe / zmienione / usunięte, statystyki
4. git add . → git commit → git push bez
   pytania o potwierdzenie
5. Podaj hash commita i liczbę plików
```

EFEKT: HISTORIA PO POLSKU



SKRÓCONA ŚCIĄGA

Nie musisz pamiętać komend. Wystarczy wiedzieć, o co poprosić agenta.

Co chcę zrobić	Co piszę agentowi	Komenda, którą rozpoznam
Sprawdzić stan	„Zrób git status”	git status
Zapisać dobrą wersję	„Zrób commit"	git add, git commit
Wysłać na GitHub	„Zrób push”	git push
Odrzucić złą zmianę przed commitem	„Przywróć zmiany z ostatniego commita.”	git restore

Agent może wykonać polecenie. Człowiek zatwierdza zakres plików, dobrą wersję i moment wysłania jej na GitHub.

CZĘŚĆ III

CWICZENIA: OD FOLDERU DO GITA

01 Komend nie musisz pamiętać

02 Agent w folderze i Git

03 Konto GitHub i logowanie agenta

04 Repozytorium i .gitignore

05 Pierwszy commit i prywatne repozytorium

KOMEND NIE MUSISZ PAMIĘTAĆ

Komendy Gita warto kojarzyć, ale nie trzeba ich znać na pamięć.

Wystarczy napisać agentowi:

> PROMPT
>
> "jaki mamy status z gitem?"

Agent sam wykona komendę i opowie, co zobaczył.

W KAŻDYM ZADANIU ZOBACZYSZ

Prompt	co piszesz agentowi
Komendę	co agent wykona, żebyś rozpoznał ją w jego odpowiedzi
Gotowe, gdy	po czym poznasz, że zadanie jest zrobione

LISTA ZADAŃ

ETAP 1 · LOKALNIE

Git i pierwszy commit na Twoim komputerze

01	Uruchom agenta w folderze projektu
02	Zainstaluj Gita
03	Skonfiguruj Gita
04	Zamień folder w repozytorium
05	Dodaj .gitignore
06	Zrób pierwszy commit z własnym opisem

ETAP 2 · Z GITHUBEM

Kopia projektu na prywatnym koncie

07	Zainstaluj GitHub CLI
08	Załóż konto na GitHubie
09	Zaloguj agenta do GitHuba
10	Utwórz prywatne repozytorium i wyślij projekt

OTWÓRZ AGENTA W FOLDERZE PROJEKTU

01 Utwórz pusty folder moj-projekt.

02 Otwórz w nim swojego agenta AI.

GOTOWE, GDY

agent podaje, że pracuje w folderze moj-projekt.

PROMPT DO AGENTA

„Powiedz, w jakim folderze pracujesz i co się w nim znajduje. Niczego nie zmieniaj.”

ZAINSTALUJ GITA

Git to osobny program. Na wielu komputerach trzeba go najpierw doinstalować.

KOMENDY DO ROZPOZNANIA

```
git --version
```

GOTOWE, GDY

agent podaje wersję Gita.

PROMPT DO AGENTA

„Sprawdź, czy mam zainstalowanego Gita. Jeżeli nie to go tutaj zainstaluj"

SKONFIGURUJ GITA

Git musi wiedzieć, kto podpisuje commity: imię i e-mail.

KOMENDY DO ROZPOZNANIA

```
git config
```

GOTOWE, GDY

agent podaje Twoje imię i e-mail ustawione w Gicie.

PROMPT DO AGENTA

„Skonfiguruj gita dla mnie: imie, email”

ZAMIEŃ FOLDER W REPOZYTORIUM

Zwykły folder staje się repozytorium — od teraz Git widzi każdą zmianę.

KOMENDY DO ROZPOZNANIA

`git init` `git status`

GOTOWE, GDY

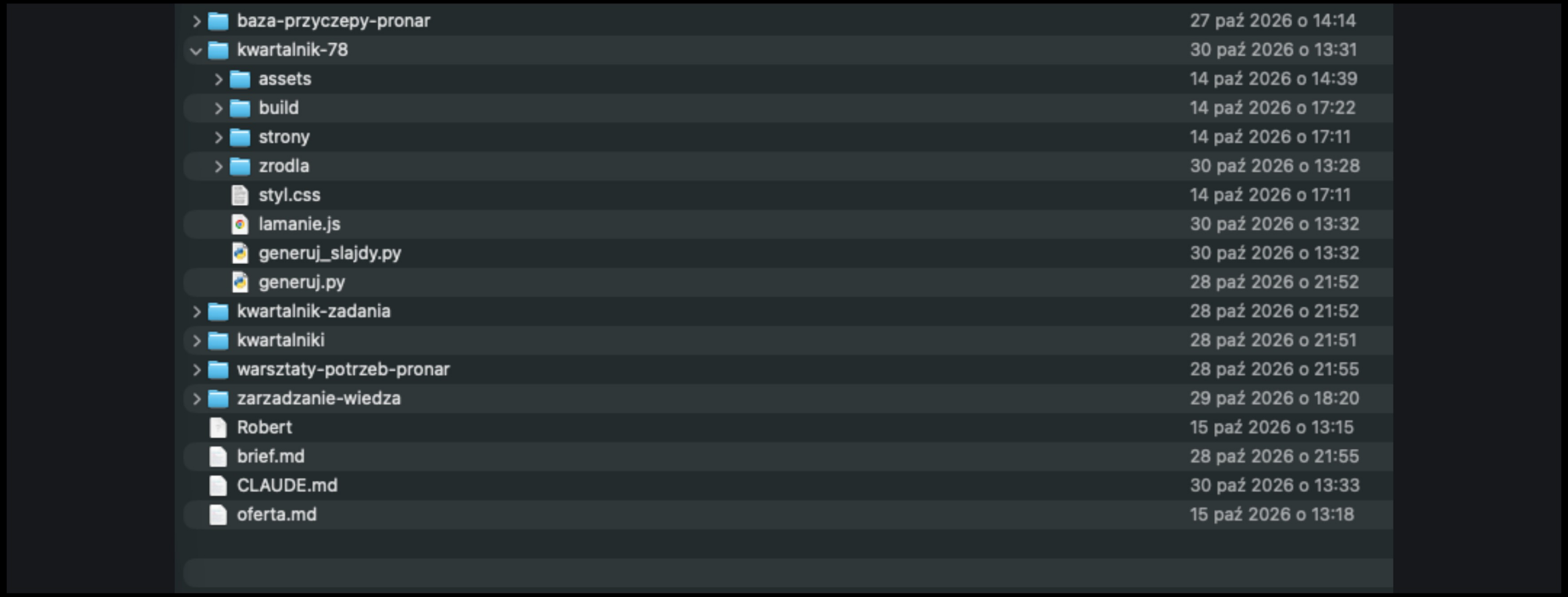
agent mówi, że repozytorium jest gotowe, a index.html i styles.css są jeszcze nieśledzone.

PROMPT DO AGENTA

„Stwórz mi tu repo git w tym folderze. Nie rób jeszcze commita. Na końcu pokaż status i wyjaśnij wynik.”

CO TRAFI DO REPOZYTORIUM?

Wskaż, które foldery trafią do repozytorium a które nie powinny



DODAJ .GITIGNORE

.gitignore to lista plików, których Git ma nie zapisywać, np.:

```
.env                    .DS_Store
node_modules/           *.log
*.mp4                   *.zip
```

GOTOWE, GDY

plik .gitignore istnieje, a status nie pokazuje .DS_Store.

Nie commitujemy haseł, tokenów, kluczy API ani * (plików z hasłami .env)

PROMPT DO AGENTA

„W git ignoruj wszelkie duze pliki np graficzne i źródłowe w tym projekcie”

ZRÓB PIERWSZY COMMIT

Pierwszy commit to pierwszy punkt, do którego zawsze można wrócić.

KOMENDY DO ROZPOZNANIA

`git add` `git commit`

OPIS COMMITA PISZESZ SAM

jedno krótkie zdanie o tym, co zawiera ta wersja.

GOTOWE, GDY

status jest czysty, a historia ma jeden commit z Twoim opisem.

PROMPT DO AGENTA

„Pokaż status, a następnie utwórz z tych zmian commit z moim opisem: »tu wpisz własny opis«”

ZAINSTALUJ GITHUB CLI

GitHub CLI (gh) to program, przez który agent pracuje z GitHubem: zakłada repozytoria i wysyła do nich zmiany.

KOMENDY DO ROZPOZNANIA

```
gh --version
```

GOTOWE, GDY

agent podaje wersję gh.

PROMPT DO AGENTA

„Sprawdź, czy jest zainstalowany GitHub CLI (gh). Jeżeli nie, zainstaluj go i pokaż wersję.”

ZAŁÓŻ KONTO NA GITHUBIE

To zadanie robisz sam, w przeglądarce.

01 Wejdź na github.com i wybierz **Sign up**.

02 Podaj e-mail, hasło i nazwę użytkownika.

03 Potwierdź adres e-mail.

04 Włącz weryfikację dwuetapową (2FA).

GOTOWE, GDY

jesteś zalogowany i widzisz swój profil.

Agent nie zakłada kont i nie zna Twojego hasła.

ZALOGUJ AGENTA DO GITHUBA

Agent łączy się z GitHubem przez GitHub CLI (gh).

Logowanie odbywa się w przeglądarce. **Hasło wpisujesz tylko tam, nigdy w rozmowie z agentem.**

KOMENDY DO ROZPOZNANIA

`gh auth login` `gh auth status`

GOTOWE, GDY

agent potwierdza, że jest zalogowany na Twoje konto.

PROMPT DO AGENTA

„Uruchom logowanie do GitHuba przez GitHub CLI w przeglądarce i powiedz mi, co mam zrobić. Na końcu sprawdź, na jakie konto jesteś zalogowany.”

UTWÓRZ PRYWATNE REPOZYTORIUM I WYŚLIJ PROJEKT

Agent zalogowany do GitHuba sam tworzy repozytorium i wysyła do niego commit.

KOMENDY DO ROZPOZNANIA

`gh repo create` `git push`

GOTOWE, GDY

na GitHubie widzisz repozytorium moj-projekt z etykietą **Private** i Twoim pierwszym commitem.

PROMPT DO AGENTA

„Utwórz na moim koncie GitHub prywatne repozytorium moj-projekt z tego folderu i wyślij do niego istniejące commity. Repozytorium musi być prywatne. Na końcu podaj link.”

CZĘŚĆ IV

PRACA W ZESPOLE NA PRZYKŁADZIE

01 clone i branch: projekt u współpracownika

02 pull request: prośba o dołączenie zmiany

03 review i merge: zmiana trafia do main

DWIE OSOBY, JEDNA STRONA

Strona matyszewski.co ma repozytorium na GitHubie. Do pracy dołącza druga osoba.

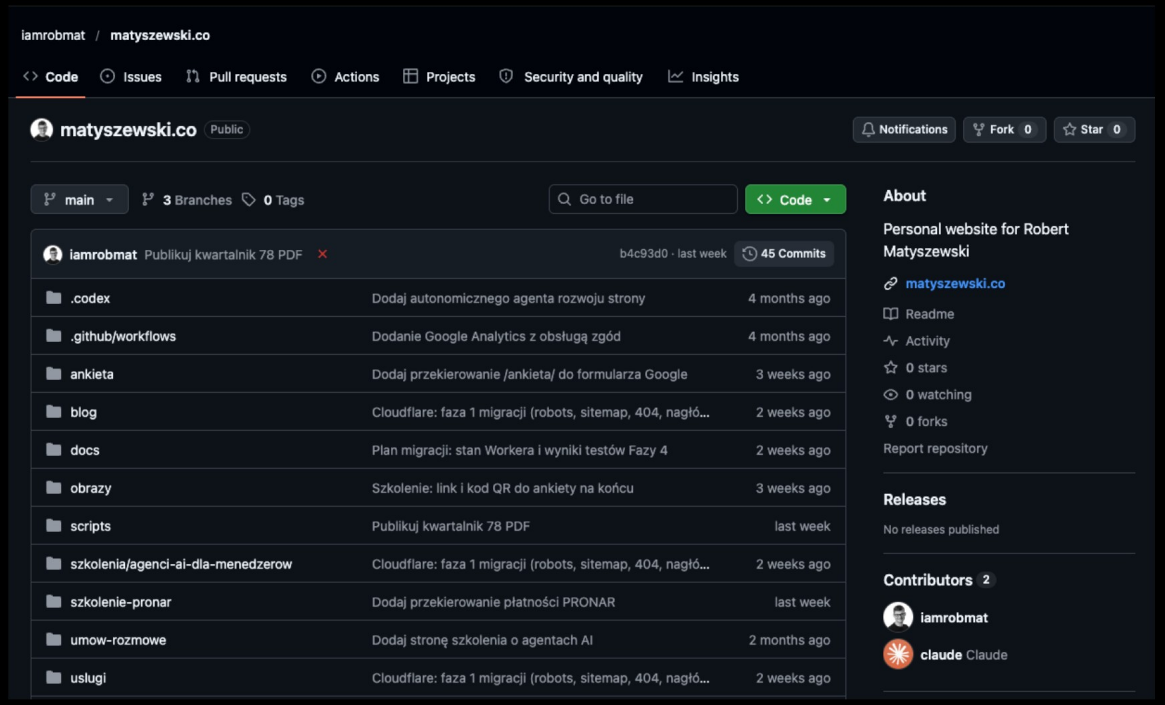
Robert

właściciel repozytorium. Sprawdza i zatwierdza zmiany.

Współpracownik

pobiera projekt, wprowadza zmianę z agentem i prosi o jej dołączenie.

Przykład: współpracownik dodaje sekcję FAQ na stronie głównej.



Gałąź main to wersja, która działa na stronie. Nikt nie zmienia jej bezpośrednio.

DROGA JEDNEJ ZMIANY

01
Dostęp
Robert zaprasza współpracownika do repozytorium

02
Clone
współpracownik pobiera projekt na swój komputer

03
Branch
tworzy osobną gałąź na zmianę

04
Commit i push
zapisuje zmianę i wysyła gałąź na GitHub

05
Pull request
prosi o dołączenie zmiany do main

06
Review
Robert sprawdza zmianę i zostawia uwagi

07
Merge
Robert dołącza zmianę do main

08
Pull
obaj pobierają nowy main przed kolejną pracą

Kroki 1, 6 i 7 robi Robert. Kroki 2–5 robi współpracownik. Krok 8 robią obaj.

ROBERT DAJE DOSTĘP

Robert robi to sam, w przeglądarce.

01 Otwiera repozytorium na GitHubie.

02 Wchodzi w **Settings** → **Collaborators**.

03 Klika **Add people** i wpisuje login współpracownika.

GOTOWE, GDY

współpracownik widzi repozytorium matyszewski.co na swoim koncie GitHub.

Współpracownik dostaje e-mail z zaproszeniem. Dostęp działa dopiero po akceptacji.

UWAGA

Współpracownik może wysyłać zmiany do każdej gałęzi, także do main. GitHub sam tego nie zablokuje. Dlatego zespół umawia się: zmiany trafiają do main tylko przez pull request.

WSPÓŁPRACOWNIK POBIERA PROJEKT

Clone tworzy na komputerze pełną kopię repozytorium: wszystkie pliki i całą historię. Robi się to raz, na początku.

KOMENDY DO ROZPOZNANIA

`git clone` `gh repo clone`

GOTOWE, GDY

agent pokazuje ostatnie commity z historii matyszewski.co.

Warunek startowy: agent jest zalogowany do GitHuba kontem współpracownika (zadanie 9 z części III).

PROMPT DO AGENTA

„Pobierz repozytorium https://github.com/iamrobmat/matyszewski.co do folderu matyszewski-co. Pokaż 5 ostatnich commitów i powiedz, na jakiej gałęzi jesteś. Niczego nie zmieniaj.”

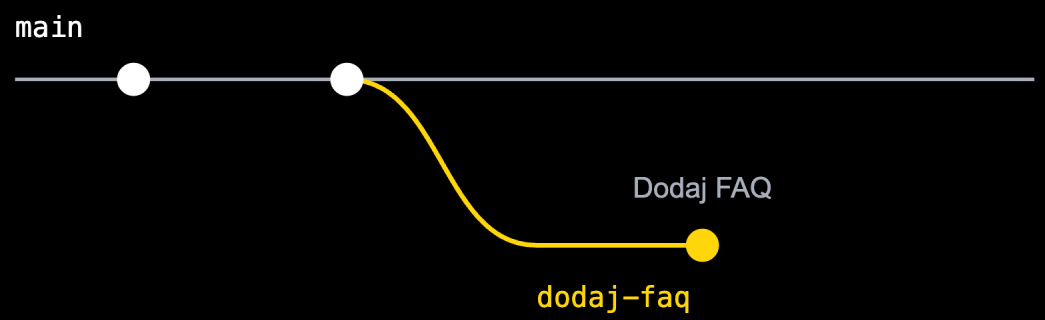
OSOBNA GAŁĄŹ NA ZMIANĘ

Branch (gałąź) to osobna linia pracy. Zmiany na gałęzi nie ruszają main, czyli działającej strony.

Jedna zmiana — jedna gałąź. Nazwa mówi, czego dotyczy: dodaj-faq.

KOMENDY DO ROZPOZNANIA

`git pull` `git switch -c`



GOTOWE, GDY

agent potwierdza, że pracujesz na gałęzi dodaj-faq.

PROMPT DO AGENTA

„Przełącz się na main i pobierz zmiany z GitHuba. Utwórz gałąź dodaj-faq i przełącz się na nią.”

ZMIANA, COMMIT I PUSH GAŁĘZI

Pracujesz jak w części II: mała zmiana, sprawdzenie, commit.

Jedna różnica: push wysyła gałąź dodaj-faq, a nie main.

KOMENDY DO ROZPOZNANIA

`git status` `git diff` `git commit` `git push`

GOTOWE, GDY

na GitHubie widać gałąź dodaj-faq z Twoim commitem.

PROMPT DO AGENTA

„Dodaj FAQ na stronie głównej, zmieniając tylko index.html i styles.css. Pokaż mi diff. Po mojej akceptacji zapisz zmianę commitem »Dodaj FAQ na stronie głównej« i wyślij gałąź dodaj-faq na GitHub.”

PULL REQUEST

Pull request (PR) to prośba: „dołącz zmiany z mojej gałęzi do main”. Powstaje na GitHubie.

PR ZBIERA W JEDNYM MIEJSCU

- tytuł i opis — co zmieniono i po co
- commity z gałęzi
- diff — zakładka **Files changed**
- komentarze i decyzję

GOTOWE, GDY

agent podaje link do pull requestu.

KOMENDY DO ROZPOZNANIA

`gh pr create`

PROMPT DO AGENTA

„Utwórz pull request z gałęzi dodaj-faq do main. Tytuł: »Dodaj FAQ na stronie głównej«. W opisie napisz krótko, co się zmieniło i jak to sprawdzić. Podaj mi link.”

ROBERT SPRAWDZA ZMIANĘ

Review to przegląd zmiany przed dołączeniem.

Robert otwiera PR i czyta diff w **Files changed**. Może komentować konkretne linijki.

NA KONIEC WYBIERA JEDNĄ Z TRZECH OPCJI	
`Comment`	same uwagi, bez decyzji
`Approve`	zmiana gotowa do dołączenia
`Request changes`	trzeba poprawić przed dołączeniem

PROMPT DO AGENTA ROBERTA

„Przejrzyj pull request »Dodaj FAQ na stronie głównej«. Wypisz, co się zmieniło, i wskaż ryzyka. Niczego nie zmieniaj i nie rób merge.”

POPRAWKI PO REVIEW

Współpracownik poprawia zmianę na tej samej gałęzi dodaj-faq.

Nowy commit i push trafiają do tego samego pull requestu. Nie zakłada się nowego PR.

KOMENDY DO ROZPOZNANIA

`git commit` `git push`

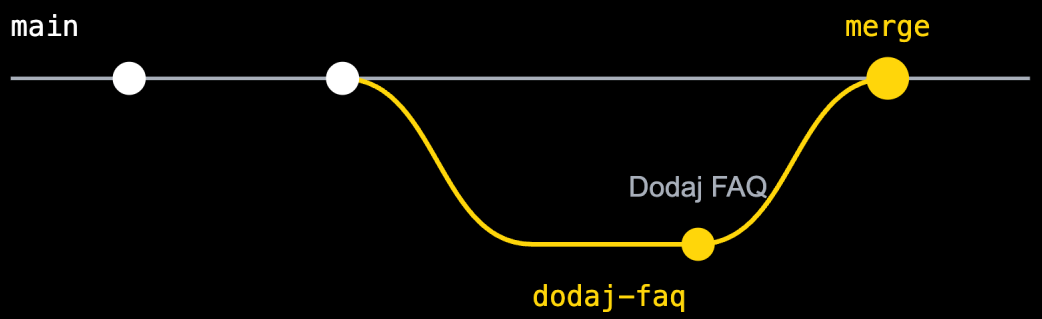
GOTOWE, GDY

w PR widać nowy commit, a Robert zatwierdza zmianę (**Approve**).

PROMPT DO AGENTA

„Przeczytaj komentarze w pull requeście »Dodaj FAQ na stronie głównej«. Wprowadź poprawki na gałęzi dodaj-faq i pokaż mi diff. Po mojej akceptacji zrób commit i push.”

MERGE: ZMIANA TRAFIA DO MAIN



Po akceptacji Robert klika na GitHubie **Merge pull request**. Zmiany z dodaj-faq trafiają do main.

Potem może usunąć gałąź przyciskiem Delete branch. Zrobiła swoje.

KONFLIKT

Pojawia się, gdy dwie gałęzie zmieniły tę samą linijkę tego samego pliku. GitHub nie pozwoli zrobić merge, dopóki konflikt nie zostanie rozwiązany.

JEŻELI PR MA KONFLIKT — WSPÓŁPRACOWNIK PISZE

„Pull request »Dodaj FAQ na stronie głównej« ma konflikt. Nie rozwiązuj go jeszcze. Wyjaśnij, które pliki i fragmenty są w konflikcie, pokaż różnice i poczekaj na moją zgodę.”

WSZYSCY MAJĄ AKTUALNY MAIN

Po merge main na GitHubie jest nowszy niż kopie na komputerach.

Pull pobiera z GitHuba nowe commity i dołącza je do Twojej gałęzi.

KOMENDY DO ROZPOZNANIA

`git switch` `git pull`

GOTOWE, GDY

w historii main widać zmianę z FAQ.

Każdy robi pull na main przed nową pracą — Robert i współpracownik.

PROMPT DO AGENTA

„Przełącz się na main, pobierz zmiany z GitHuba i pokaż 3 ostatnie commity. Jeśli gałąź dodaj-faq jest już dołączona, usuń jej lokalną kopię.”